

Công nghệ phần mềm

Chương 04: Đảm bảo chất lượng và kiểm thử phần mềm



Giảng viên: Lê Thị Hoàng Anh

Email: anhlth@nuce.edu.vn



Nội dung

1. Đảm bảo chất lượng phần mềm

- Các nhân tố chất lượng phần mềm
- Độ đo chất lượng phần mềm
- Độ tin cậy phần mềm
- Cách tiếp cận đảm bảo chất lượng phần mềm

2. Kiểm thử phần mềm

- Nền tảng của kiểm thử phần mềm
- Chiến lược kiểm thử phần mềm



Đảm bảo chất lượng phần mềm

- **Chất lượng là gì?**
 - Là khả năng của tập hợp các đặc tính của một sản phẩm/hệ thống hay quá trình,
 - để đáp ứng yêu cầu của khách hàng hay các bên liên quan.
- **Đảm bảo chất lượng: Liên quan đến việc**
 - Đảm bảo một sản phẩm phần mềm đạt được mức độ yêu cầu về chất lượng.
 - Định nghĩa các tiêu chuẩn chất lượng phù hợp và đảm bảo rằng những điều này được tuân thủ.



Các nhân tố chất lượng phần mềm

- Là cái mốc để đánh giá chất lượng phần mềm. Tập trung vào ba khía cạnh:
 - Các đặc trưng vận hành
 - Khả năng trải qua thay đổi
 - Tính thích nghi với môi trường mới
- Một sản phẩm phần mềm cần xét theo các tiêu chuẩn:
 - Tính đúng đắn, tính khoa học, tính hữu hiệu, tính sáng tạo, tính an toàn, tính toàn vẹn, tính đầy đủ, tính độc lập, tính phổ dụng, tính đơn giản, tính dễ phát triển để mở rộng...



Độ đo chất lượng phần mềm

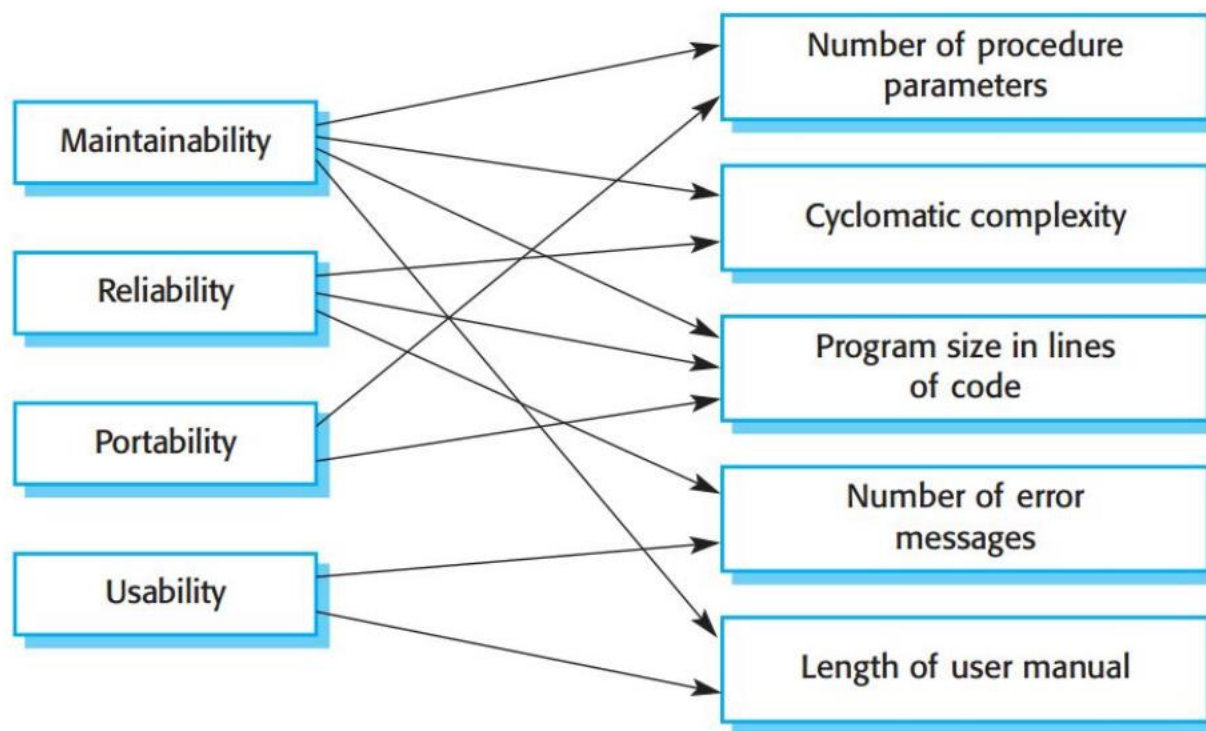
- Các độ đo phần mềm được dùng cho việc thẩm định mang tính định lượng:
 - Chỉ số chất lượng phần mềm
 - Khoa học phần mềm HALSTEAD
 - Độ đo phức tạp của Thomas McCabe(Xem thêm trong sách giáo trình Trường Đại Học Xây Dựng Hà Nội)



Độ đo chất lượng phần mềm

- Theo chuẩn ISO/IEC 25010 ta có thể đo lường phần mềm theo 8 độ đo

<https://www.sealights.io/software-quality/software-quality-measurement-8-dimensions-of-quality-and-how-to-measure-them/>





Độ đo chung

- Một số độ đo

Metrics

Fan-in/Fan-out

Chiều dài đoạn mã (LOC – Line of Code)

Chiều dài của định danh (LI – Length of identifiers)

Độ sâu của các lệnh lồng có điều kiện

Chỉ số Fog (Fog index)

Độ phức tạp chu kỳ (CC - Cyclomatic complexity)



Độ đo hướng đối tượng

- Một số độ đo

OO Metrics
Độ sâu của cây thừa kế (DIT – Depth of Inheritance Tree)
Số lượng lớp con trực tiếp (NOC – Number of Children)
Phương thức trọng số cho lớp (WMC – Weighted methods per class)
Coupling between object classes (CBO – Coupling between object classes)
Số lượng phương thức ghi đè (NOM – Number Of Overridden Methods)



Độ tin cậy phần mềm

- Độ tin cậy phần mềm có thể được đo trực tiếp và ước lượng bằng cách dùng dữ liệu lịch sử và đang phát triển.
- Được định nghĩa dưới dạng xác suất thống kê:
 - Số lần vận hành không thất bại/ tổng số lần
 - Xét trong một khoảng thời gian xác định



Độ tin cậy phần mềm

- Cách đo đơn giản về độ tin cậy là thời gian trung bình giữa những lần thất bại:

$$MTBF = MTTF + MTTR$$

– Trong đó:

- MTTF là thời gian trung bình của thất bại
- MTTR là thời gian trung bình để sửa chữa tương ứng



Độ tin cậy phần mềm

- Ngoài độ tin cậy chúng ta cần đo tính có sẵn của phần mềm.
 - Là xác suất một chương trình vận hành theo yêu cầu tại một điểm nào đó theo thời gian

$$\text{Tính sẵn có} = \frac{MTTF}{(MTTF + MTTR)} \times 100\%$$



Cách tiếp cận đảm bảo chất lượng phần mềm

- Đảm bảo chất lượng phần mềm SQA là một mẫu hình hành động có hệ thống và có kế hoạch.
- Bao gồm 7 hoạt động:
 - (1) Áp dụng các phương pháp kỹ thuật, (2) Tiến hành các xét duyệt kỹ thuật chính thức, (3) Kiểm thử phần mềm, (4) Buộc tôn trọng các chuẩn, (5) Kiểm soát thay đổi, (6) Đo, (7) Lưu giữ và báo cáo kết quả ghi lại.



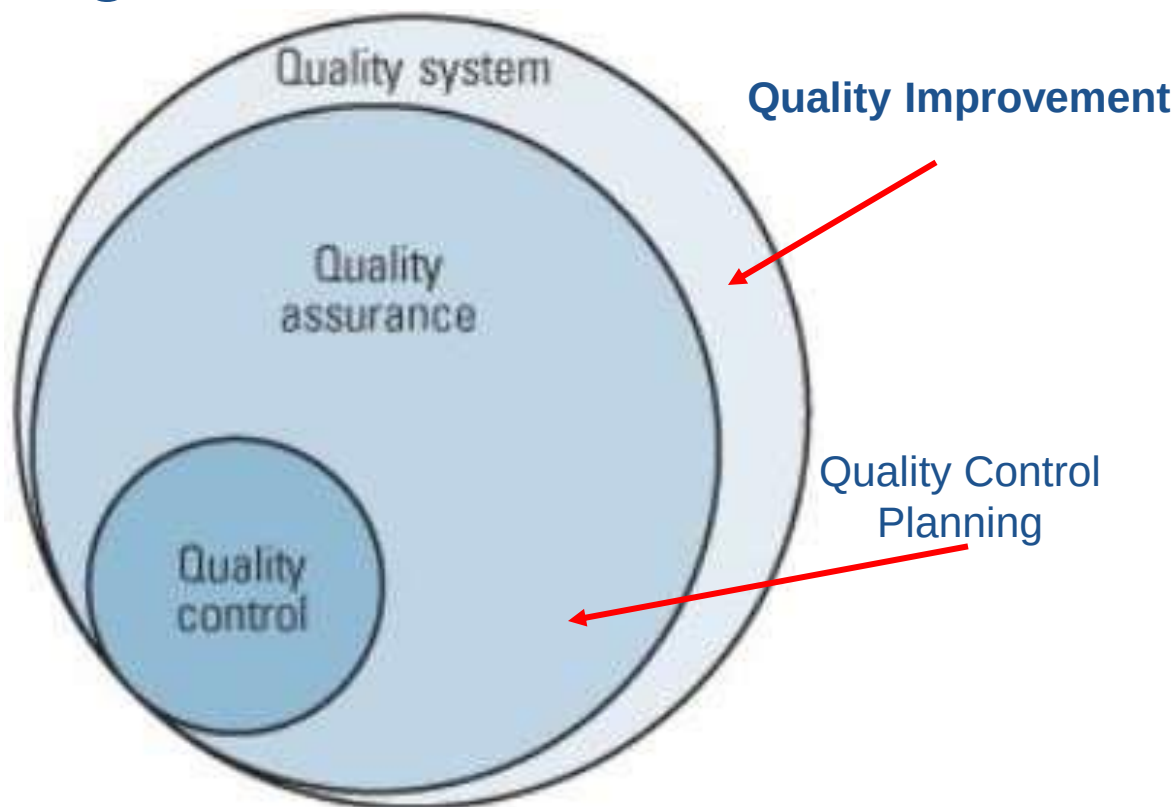
Quản lý chất lượng phần mềm

- Là một quy trình:
 - Áp dụng lên suốt dự án và vòng đời phần mềm, chứ không phải là
 - hoạt động đơn lẻ
 - tại một thời điểm nhất định.
- Một số yếu tố thành phần của quản lý chất lượng:
 - Kiểm tra chất lượng (QA)
 - Lập kế hoạch chất lượng (QP)
 - Kiểm soát chất lượng (QC)
 - Cải tiến chất lượng (QI)



Hoạt động quản lý chất lượng

- Bốn thành phần chính của hệ thống quản lý chất lượng





Hoạt động quản lý chất lượng

1. Quality Assurance (QA)

- Thiết lập quy trình và tiêu chuẩn chất lượng

2. Quality Planning (QP)

- Lựa chọn các thuộc tính chất lượng, lập kế hoạch quản lý chất lượng

3. Quality Control (QC)

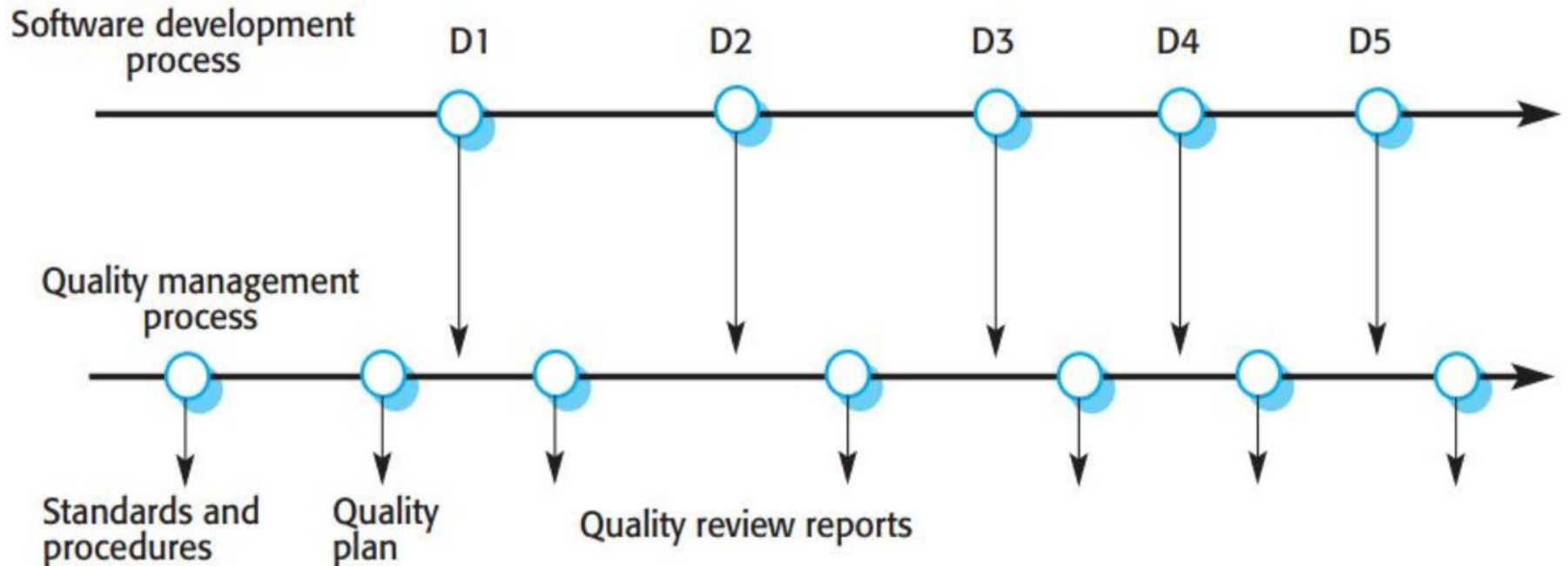
- Thực hiện đảm bảo các quy trình và tiêu chuẩn được tuân thủ

Quản lý chất lượng nên tách biệt với quản lý dự án để đảm bảo sự độc lập



Quản lý chất lượng

- Quản lý chất lượng & phát triển PM





Quality Planning

- Lập kế hoạch chất lượng:
 - Là phương pháp quyết định điều gì là tối quan trọng với dự án trong giai đoạn lập kế hoạch dự án.
 - Nó chỉ ra cách đo chất lượng dự án để ngừa lỗi
 - Nhằm mục đích xác định:
 - Các tiêu chuẩn chất lượng có liên quan
 - Kết hợp các tiêu chuẩn chất lượng vào sản phẩm và quy trình.
 - Xác định cách thức để đạt được các tiêu chuẩn này.



Quality Planning

- Các hoạt động lập kế hoạch chất lượng
 - Xác định đầu vào cho QP
 - Xác định kỳ vọng tiêu chuẩn, lựa chọn các thuộc tính chất lượng của công ty.
 - Xác định phạm vi dự án (Nhân sự/thời gian/ngân sách...)
 - Các đầu vào khác như tài liệu mô tả/thiết kế sản phẩm, các tiêu chuẩn quy định ảnh hưởng đến dự án (ví dụ chính sách lương thưởng, KPI ...)



Quality Planning

- Các hoạt động lập kế hoạch chất lượng (Tiếp)
 - Đo lường chất lượng bằng cách sử dụng một số công cụ (tool). Mục đích:
 - Lập ước tính, theo dõi tiến độ của dự án
 - Phân tích các khiếm khuyết
 - Đạt được sự cải tiến liên tục
 - Lập kế hoạch chất lượng.



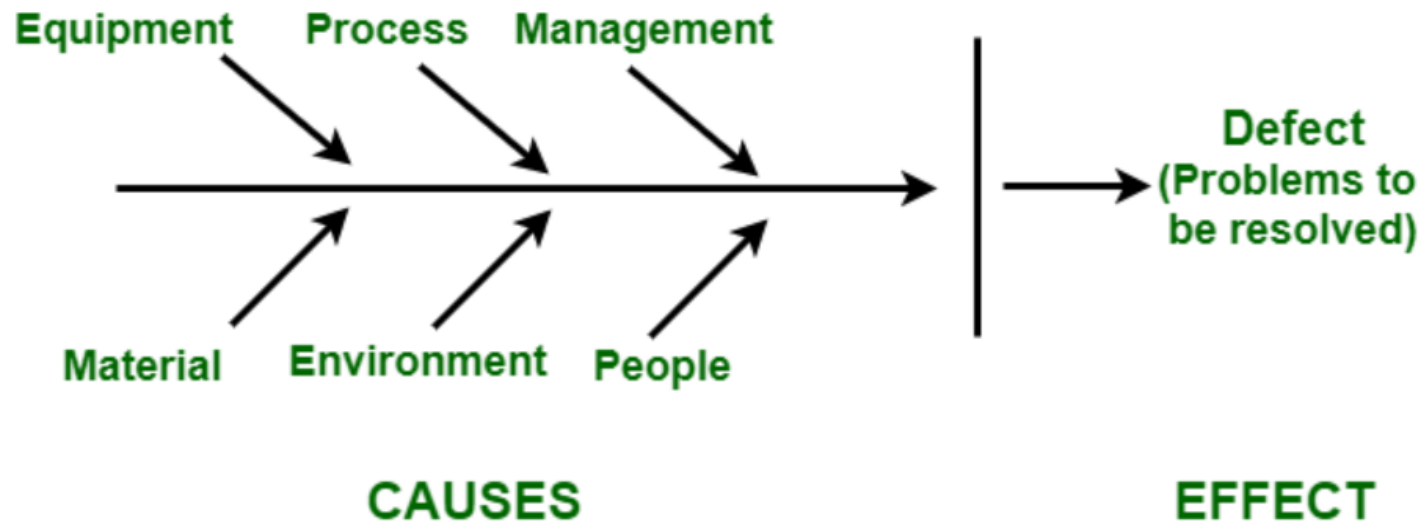
Quality Planning

- Các hoạt động lập kế hoạch chất lượng (Tiếp)
 - Một số kỹ thuật sử dụng để lập kế hoạch CL:
 - Phân tích lợi ích-chi phí (Cost-Benefit Analysis)
 - Chi phí khi tuân thủ
 - Chi phí khi gặp lỗi: Lỗi bên trong, lỗi bên ngoài
 - Đo điểm chuẩn (Benchmarking)
 - Tạo sơ đồ
 - Sơ đồ luồng hệ thống hoặc quy trình
 - Sơ đồ nhân quả (Sơ đồ xương cá của Ishikawa)
 - Check sheets, Biểu đồ Pareto, Control Charts, Histogram...
 - Thiết kế thử nghiệm



Quality Planning

- Nguồn: [Geeksforgeeks.org](https://www.geeksforgeeks.org/)



Fishbone diagram

Sơ đồ xương cá Kaoru Ishikawa



Quality Control

- Kiểm soát chất lượng:
 - Là việc kiểm tra, đánh giá chất lượng phần mềm theo đúng quy trình và tiêu chuẩn đang được tuân thủ/ thiết lập trong QA.
- Hai phương pháp để kiểm soát chất lượng:
 - Duyệt/rà soát lại chất lượng
 - Thực thi công việc đo lường phần mềm thông qua các độ đo



Quality Control

- QC1: Các kiểu duyệt lại để kiểm soát chất lượng
 - Duyệt lại để loại bỏ khiếm khuyết (SP)
 - Duyệt lại về tiến độ (Sản Phẩm+Quy Trình)
 - Duyệt lại chất lượng (SP+ Tiêu chuẩn)



Quality Control

- QC2: Thực thi đo lường phần mềm thông qua các độ đo:
 - Đo lường phần mềm liên quan đến việc tìm ra một giá trị bằng số cho một thuộc tính của sản phẩm phần mềm hoặc quy trình.
 - Cho phép so sánh khách quan các kỹ thuật và quy trình khác nhau.
 - Độ đo: Là bất kỳ loại phép đo nào liên quan đến hệ thống phần mềm, quy trình hoặc tài liệu
 - Số dòng mã trong chương trình, chỉ số Fog...
 - Cho phép lượng hóa phần mềm và quy trình phần mềm.



Nội dung

1. Đảm bảo chất lượng phần mềm

- Các nhân tố chất lượng phần mềm
- Độ đo chất lượng phần mềm
- Độ tin cậy phần mềm
- Cách tiếp cận đảm bảo chất lượng phần mềm

→ 2. Kiểm thử phần mềm

- Nền tảng của kiểm thử phần mềm
- Chiến lược kiểm thử phần mềm



Một số định nghĩa về kiểm thử

- Kiểm thử phần mềm là quá trình thực thi phần mềm với mục tiêu tìm ra lỗi

Glen Myers, 1979

→ Khẳng định được chất lượng của phần mềm đang xây dựng

Hetzel, 1988

- Kiểm thử là một phần của quy trình thẩm định và kiểm định phần mềm (verification and validation – V&V).



Một số đặc điểm của kiểm thử PM

- Kiểm thử phần mềm giúp tìm ra được sự hiện diện của lỗi nhưng không thể chỉ ra sự vắng mặt của lỗi

Dijkstra

- Mọi phương pháp được dùng để ngăn ngừa hoặc tìm ra lỗi đều sót lại những lỗi khó phát hiện hơn

Beizer

- Điều gì xảy ra nếu việc kiểm thử không tìm được lỗi trong phần mềm hoặc phát hiện quá ít lỗi
 - Phần mềm có chất lượng quá tốt
 - Quy trình/Đội ngũ kiểm thử hoạt động không hiệu quả



6 điểm lưu ý khi kiểm thử

- (1) Chất lượng phần mềm do khâu phân tích, thiết kế quyết định là chủ yếu chứ không phải khâu kiểm thử.
- (2) Tính dễ kiểm thử phụ thuộc vào cấu trúc chương trình
- (3) Người kiểm thử và người phát triển nên khác nhau



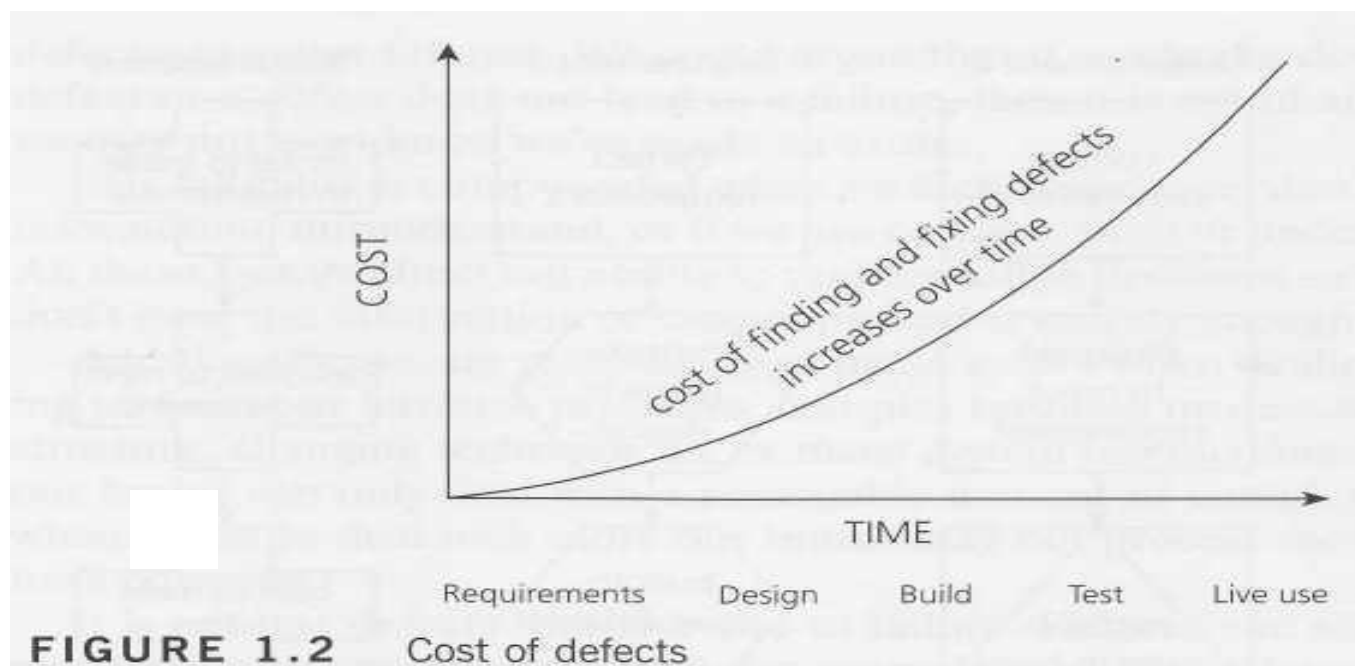
6 điểm lưu ý khi kiểm thử (tiếp)

- (4) Dữ liệu thử cho kết quả bình thường thì không có ý nghĩa nhiều, cần có những dữ liệu kiểm thử mà phát hiện ra lỗi
- (5) Khi thiết kế trường hợp thử, không chỉ dữ liệu kiểm thử nhập vào, mà phải thiết kế trước cả dữ liệu kết quả sẽ có
- (6) Khi phát sinh thêm trường hợp thử thì nên thử lại những trường hợp thử trước đó để tránh ảnh hưởng lan truyền sóng



Chi phí tìm lỗi tăng lên khi nào?

- Chi phí cho việc tìm thấy và sửa lỗi **tăng dần** trong suốt chu kỳ sống của phần mềm. Lỗi tìm thấy **càng sớm** thì **chi phí để sửa càng thấp** và ngược lại.





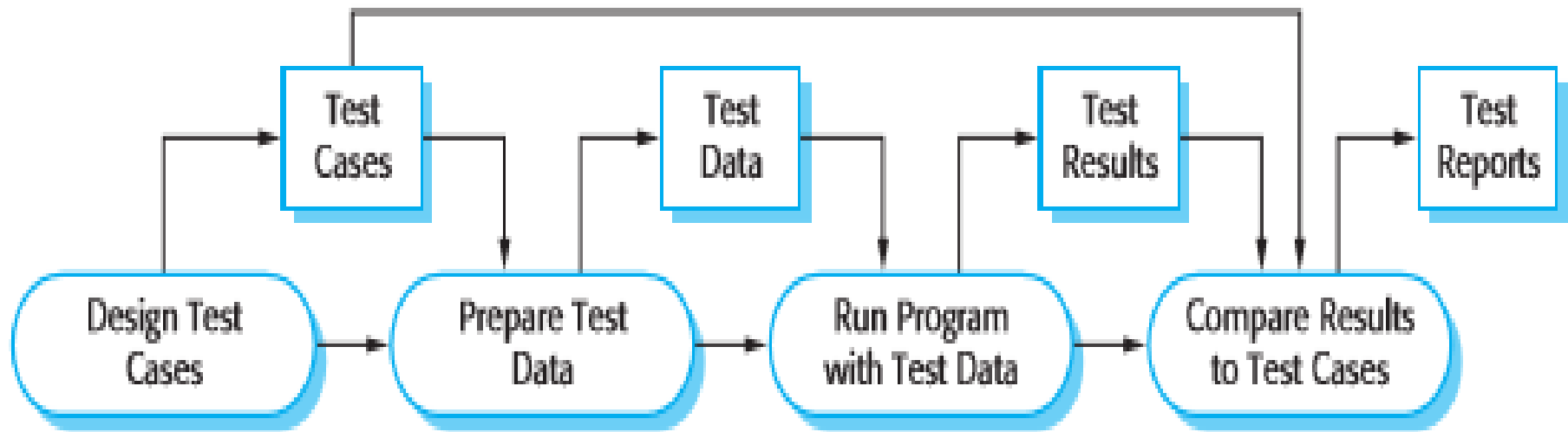
Thời điểm tiến hành kiểm thử

Tiến hành ở mọi công đoạn phát triển phần mềm

- ❑ phân tích
 - xét duyệt đặc tả yêu cầu
- ❑ thiết kế
 - xét duyệt đặc tả thiết kế
- ❑ mã hóa
 - kiểm thử chương trình



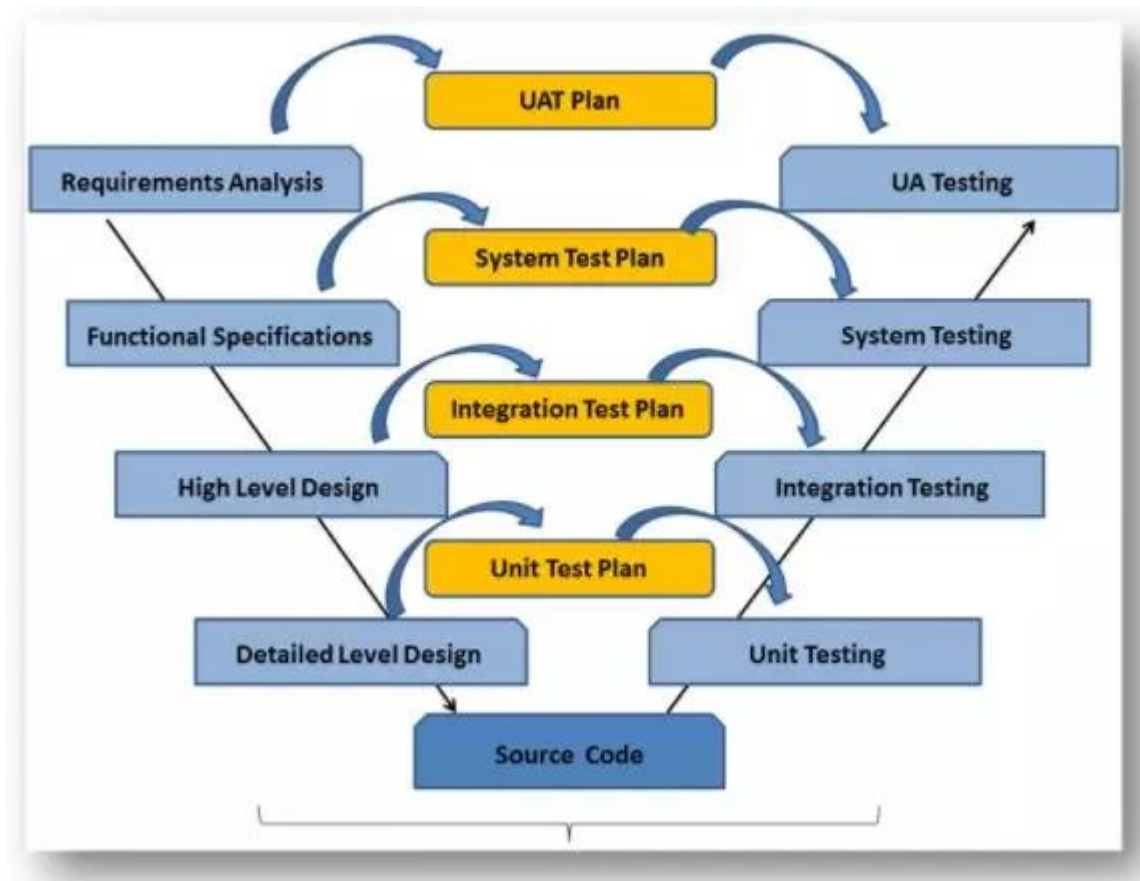
Một mô hình quy trình kiểm thử phần mềm





Tương ứng giữa vòng đời dự án và kiểm thử

- Mô hình chữ V





Nguyên tắc kiểm thử

- Chọn các input làm cho hệ thống tạo ra tất cả các thông báo lỗi;
- Thiết kế input làm tràn bộ đệm;
- Lặp lại cùng một input hay một dãy các input một vài lần;
- Ép các output không hợp lệ phải xuất hiện;
- Ép các kết quả tính toán phải hoặc là quá lớn hoặc là quá nhỏ.



Quan điểm kiểm thử

- Kiểm thử chỉ ra sự hiện diện của lỗi
- Kiểm thử toàn diện đầy đủ là không thể
- Cần bắt đầu giai đoạn kiểm thử càng sớm càng tốt
- Phân nhóm lỗi để xác định một số module tập trung lỗi nhiều nhất
- Pesticide paradox
- Kiểm thử được thực hiện khác nhau trong những bối cảnh khác nhau
- Suy nghĩ không có lỗi là một sai lầm



Chính sách kiểm thử (Testing Policy)

- Kiểm tra tất cả các chức năng trong hệ thống menu.
- Kiểm tra tất cả các **mục khác** có cùng chức năng trong hệ thống menu (Toolbar, Listbar, Dialog bar, Context Menu,...)
- Kiểm tra cùng một chức năng với nhiều vai trò khác nhau (đối với hệ thống có nhiều người dùng)
- Kiểm tra tất cả các **dữ liệu bắt buộc nhập** trong các màn hình (hợp lệ/không hợp lệ)



Các giai đoạn của kiểm thử

- Kiểm thử trong khi xây dựng (Development testing)
Hệ thống được kiểm thử trong quá trình phát triển để tìm ra lỗi.
- Kiểm thử bản release (release testing)
Một đội ngũ kiểm thử độc lập sẽ kiểm thử một phiên bản hoàn chỉnh của hệ thống trước khi nó được bàn giao cho người dùng.
- Kiểm thử người dùng (user testing)
Người dùng hoặc người dùng tiềm năng của hệ thống kiểm thử hệ thống trên môi trường của họ



Phân loại

- Kiểm thử trong khi xây dựng: Gồm tất cả các hoạt động kiểm thử được tiến hành bởi nhóm phát triển hệ thống.
 - Kiểm thử đơn vị (unit testing)
 - Kiểm thử component (component testing)
 - Kiểm thử tích hợp (intergration testing)
 - Kiểm thử hệ thống (system testing)
- Kiểm thử bản release: Kiểm thử để chuẩn bị phát hành(validation test và defect test).
 - Scenario based testing – kiểm thử theo kịch bản (use case)
 - Performance testing – kiểm thử hiệu năng
- Kiểm thử người dùng(user testing)
 - Kiểm thử người dùng gồm: Acceptance test, Alpha,Beta testing



Kiểm thử đơn vị trong OOP

- Là quy trình kiểm thử từng thành phần riêng lẻ.
- Đây là quy trình kiểm thử tìm lỗi.
- Các đơn vị có thể là:
 - Các hàm hay phương thức đơn lẻ trong một lớp đối tượng (class).
 - Các lớp đối tượng (class) chứa vài thuộc tính và phương thức.
 - Các thành phần với các giao diện được định nghĩa sẵn để truy cập vào các tính năng của chúng.



Kiểm thử đơn vị trong OOP

- Người tiến hành thường là người lập trình
- Sử dụng các stubs, mock, fake và drivers
- Phối hợp giữa kiểm thử cấu trúc và kiểm thử chức năng
 - Kiểm tra câu lệnh điều kiện, cấu trúc dữ liệu điều kiện biên,...
- Tài liệu thường sơ sài



Kiểm thử thành phần trong OOP (component testing)

- Kiểm thử thành phần là quá trình kiểm thử các thành phần riêng biệt của hệ thống.
- Với hầu hết các hệ thống, người phát triển các thành phần chịu trách nhiệm kiểm thử các thành phần.
- Ta có thể kiểm thử chúng theo các bước sau:
 1. Các chức năng và cách thức riêng biệt bên trong đối tượng.
 2. Các lớp đối tượng có một vài thuộc tính và phương thức.
 3. Kết hợp các thành phần **để tạo nên các đối tượng và chức năng khác nhau**. Các thành phần hỗn hợp có một giao diện rõ ràng được sử dụng để truy cập các chức năng của chúng.



Kiểm thử tích hợp trong OOP

- Quá trình kiểm thử tích hợp bao gồm việc xây dựng hệ thống từ các thành phần và kiểm thử hệ thống tổng hợp với các vấn đề phát sinh từ sự tương tác giữa các thành phần.
- Kiểm thử tích hợp kiểm tra trên thực tế các thành phần làm việc với nhau, được gọi là chính xác và truyền dữ liệu đúng, vào lúc thời gian đúng, thông qua giao diện của chúng.
- Hệ thống tích hợp bao gồm một nhóm các thành phần thực hiện vài chức năng của hệ thống và được tích hợp với nhau bằng cách gộp các mã. Tích hợp theo hai cách là bottom-up hay top-down...



Kiểm thử tích hợp trong OOP

- Trong Unit Test, lập trình viên cố gắng phát hiện lỗi liên quan đến chức năng và cấu trúc nội tại của Unit. Có một số phép kiểm thử đơn giản trên giao tiếp giữa Unit với các thành phần liên quan khác, tuy nhiên mọi giao tiếp liên quan đến Unit chỉ thật sự được kiểm tra đầy đủ khi các Unit tích hợp với nhau trong khi thực hiện Integration Test.
- Trừ một số ít ngoại lệ, Integration Test chỉ nên thực hiện trên những Unit đã được kiểm tra cẩn thận trước đó bằng Unit Test, và tất cả các lỗi mức Unit đã được sửa chữa.



Kiểm thử tích hợp trong OOP

- Có 4 loại kiểm tra trong KTTH:
 - Kiểm tra cấu trúc
 - Kiểm tra chức năng
 - Kiểm tra hiệu năng
 - Kiểm tra khả năng chịu tải



Kiểm thử hệ thống

- Mục đích là kiểm thử thiết kế và toàn bộ hệ thống (sau khi tích hợp) có thỏa mãn yêu cầu đặt ra hay không.
- Trong nhiều trường hợp, đòi hỏi một số thiết bị phụ trợ, phần mềm hoặc phần cứng đặc thù.
- Với hầu hết các hệ thống phức tạp, kiểm thử hệ thống gồm hai giai đoạn riêng biệt:
 1. Kiểm thử tích hợp: đội kiểm thử nhận mã nguồn của hệ thống và nhận biết thành phần cần phải gỡ lỗi. Kiểm thử tích hợp hầu như liên quan với việc tìm các khiếm khuyết của hệ thống.
 2. Kiểm thử phát hành: Một phiên bản của hệ thống có thể được phát hành tới người dùng được kiểm thử, thường là kiểm thử “hộp đen”. Các vấn đề được báo cáo cho đội phát triển để gỡ lỗi chương trình. Khách hàng được bao hàm trong kiểm thử phát hành, thường được gọi là kiểm thử chấp nhận.



Kiểm thử bản release

- Là quy trình kiểm thử một bản release của hệ thống, bản này sẽ sử dụng bên ngoài đội ngũ phát triển hệ thống.
- Mục tiêu chính là để thuyết phục khách hàng rằng hệ thống đủ tốt để đưa vào sử dụng.
 - Phải chỉ ra được rằng hệ thống hỗ trợ các tính năng đã đặc tả, đảm bảo hiệu năng và độ tin cậy, và không có lỗi khi sử dụng.
- Là quy trình kiểm thử hộp đen trong đó các test chỉ bắt nguồn từ đặc tả hệ thống.



Kiểm thử bản release

- Kiểm thử bản release là một hình thức của kiểm thử hệ thống.
- Điểm khác nhau quan trọng:
 - Một nhóm tách biệt không tham gia vào việc phát triển sẽ chịu trách nhiệm về kiểm thử bản release.
 - Kiểm thử hệ thống bởi nhóm phát triển nên tập trung vào việc tìm lỗi trong hệ thống (defect testing).
 - Mục tiêu của kiểm thử bản release là để chứng tỏ rằng hệ thống đáp ứng yêu cầu và đủ tốt để đưa ra sử dụng bên ngoài (validation testing).



Acceptance Test

- Có sự tham gia của khách hàng/người sử dụng
- Dùng kiểm thử chức năng
- Mục đích: thẩm định(validation) phần mềm – sai sót, thiếu sót - so với yêu cầu người dùng
- Sử dụng các dữ liệu thực do user cung cấp



Kiểm thử người dùng

- Alpha testing:
 - Kiểm thử chấp nhận tiến hành ở môi trường khách hàng được gọi là alpha testing
- Beta testing:
 - Mở rộng của alpha testing
 - Được tiến hành với một lượng lớn users
 - User tiến hành kiểm thử không có sự hướng dẫn của người phát triển
 - Thông báo lại kết quả cho người phát triển



Phân loại theo chiến lược kiểm thử (strategy)

- Kiểm nghiệm tầm hợp: Kiểm nghiệm các bộ phận riêng rẽ, bao gồm:

White-box testing :

- Component hay Unit Testing,
- Object class testing

Black-box testing:

- Functional testing, Interface testing, Alpha testing , Beta testing, Release testing, Stress testing, Ad hoc testing, Performance testing...

Gray-box testing:

- Trong Kiểm thử Hộp xám, cấu trúc bên trong sản phẩm chỉ được biết một phần, Tester có thể truy cập vào cấu trúc dữ liệu bên trong và thuật toán của chương trình với mục đích là để thiết kế các test case, nhưng khi test thì test như là người dùng cuối hoặc là ở mức hộp đen.



Phân loại theo chiến lược kiểm thử (strategy)

- Kiểm nghiệm tầm rộng:
 - Kiểm nghiệm bộ phận (Module testing): kiểm nghiệm một bộ phận riêng rẽ.
 - Kiểm nghiệm tích hợp (Integration testing): tích hợp các bộ phận và hệ thống con.
 - Kiểm nghiệm hệ thống (System testing): kiểm nghiệm toàn bộ hệ thống.
 - Kiểm nghiệm chấp nhận (Acceptance testing): thực hiện bởi khách hàng.



Một số kỹ thuật test

- Test tĩnh:
 - Dựa vào việc kiểm tra tài liệu, source code,... mà không cần phải thực thi phần mềm.
 - Các lỗi được tìm thấy trong quá trình kiểm tra có thể dễ dàng được loại bỏ và chi phí rẻ hơn nhiều so với khi tìm thấy trong test động. Một số lợi ích khi thực hiện việc kiểm tra (reviews):
 - Lỗi sớm được tìm thấy và sửa chữa
 - Giảm thời gian lập trình
 - Giảm thời gian và chi phí test



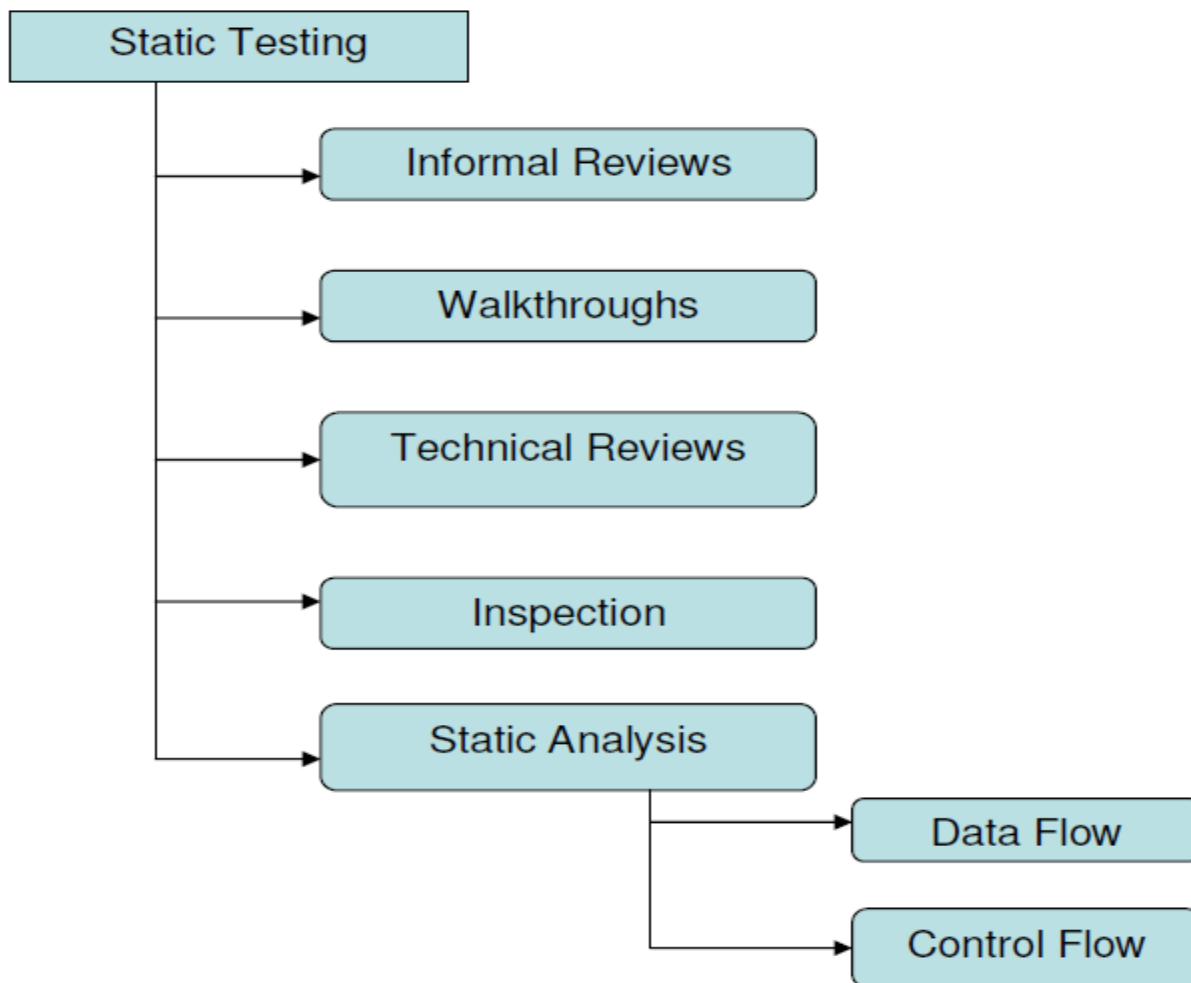
Một số kỹ thuật test

- Test tĩnh (tiếp):
 - Các tài liệu được kiểm thử:
 - Tài liệu đặc tả yêu cầu
 - Tài liệu đặc tả thiết kế
 - Sơ đồ luồng dữ liệu
 - Mô hình ER
 - Source code
 - Test case
 - ...



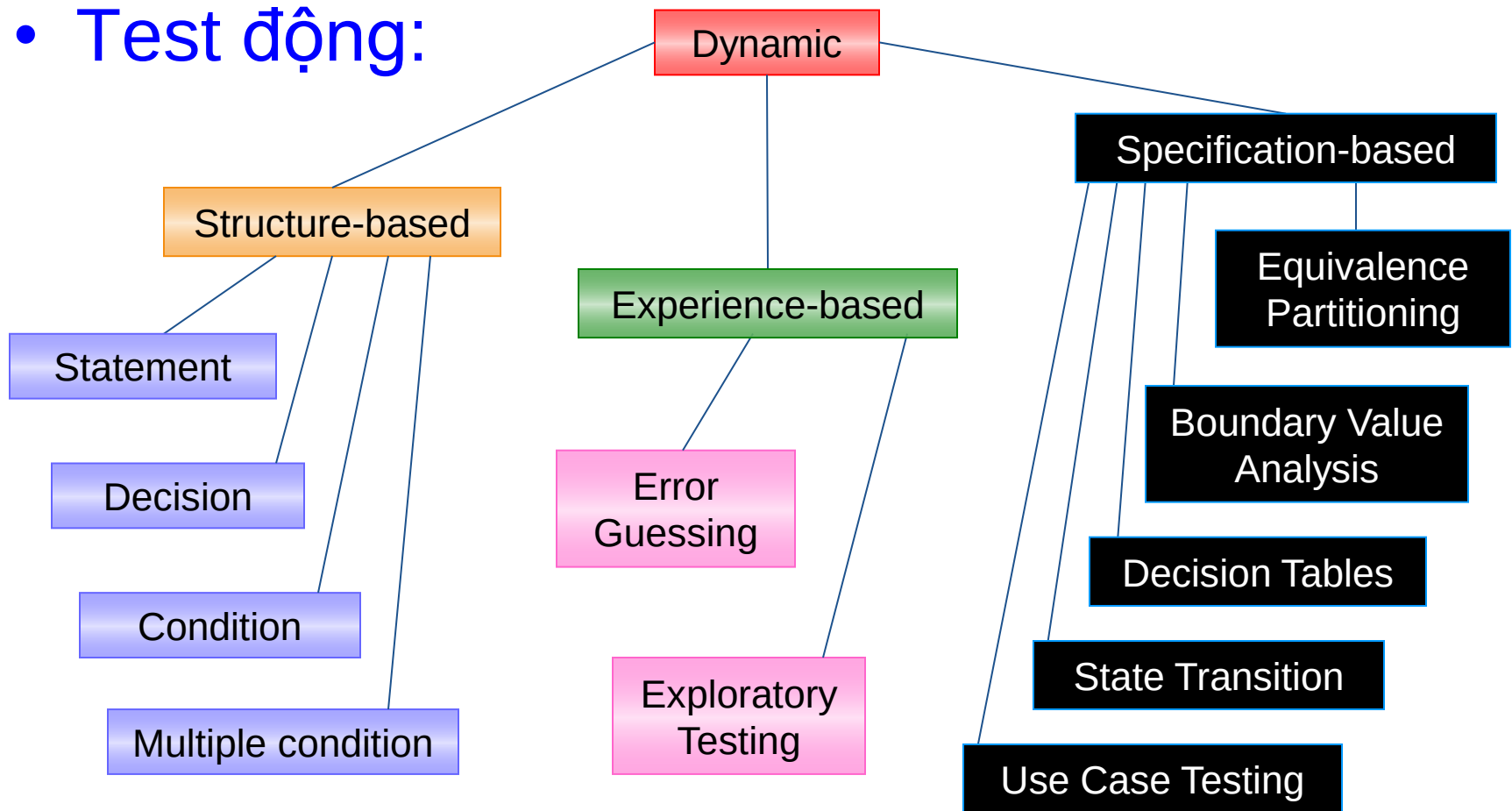
Các kỹ thuật trong kiểm thử tĩnh

- Tổng quan



Một số kỹ thuật test

- Test động:





Một số kỹ thuật test

- Test động:
 - Test dựa trên mô tả (specification-based) hay còn gọi test chức năng (functional testing): Test những gì mà phần mềm phải làm, không cần biết phần mềm làm như thế nào (kỹ thuật **black box**)
 - Test dựa trên cấu trúc (structure-based) hay còn gọi test phi chức năng (non-functional testing): Test phần mềm hoạt động như thế nào (kỹ thuật **white box**)
 - Test dựa trên kinh nghiệm (experience-based): đòi hỏi sự hiểu biết, kỹ năng và kinh nghiệm của người test



Các kỹ thuật trong kiểm thử hộp đen

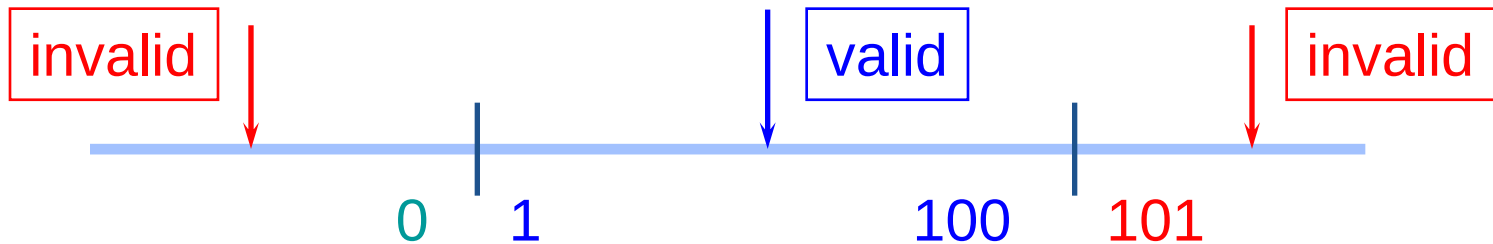
- Kỹ thuật phân vùng tương đương – EP (Equivalence Partitioning)
 - Ví dụ: một textbox chỉ cho phép nhập số nguyên từ 1 đến 100
 - → Ta không thể nhập tất cả các giá trị từ 1 đến 100
 - Ý tưởng của kỹ thuật này: chia (partition) đầu vào thành những nhóm tương đương nhau (equivalence). Nếu một giá trị trong nhóm hoạt động đúng thì tất cả các giá trị trong nhóm đó cũng hoạt động đúng và ngược lại.



Các kỹ thuật trong kiểm thử hộp đen

- Kỹ thuật phân vùng tương đương – EP (tiếp)

- Trong ví dụ trên dùng kỹ thuật phân vùng tương đương, chia làm 3 phân vùng như sau:



- Như vậy chỉ cần chọn 3 test case để test trường hợp này: -5, 55, 102 hoặc 0, 10, 1000, ...



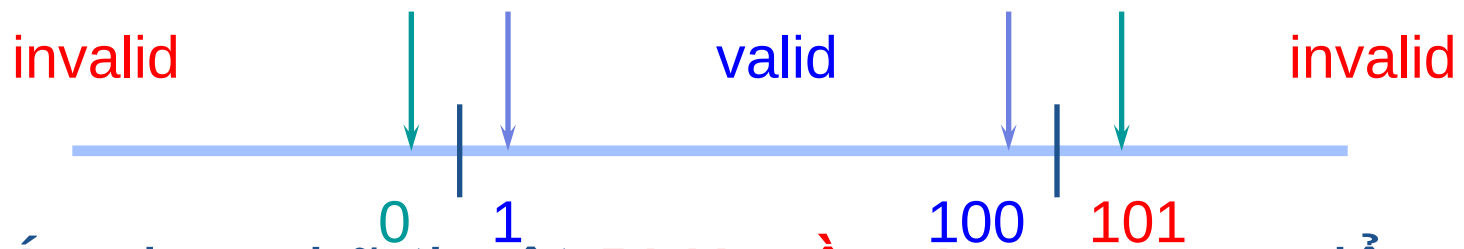
Các kỹ thuật trong kiểm thử hộp đen

- Kỹ thuật phân vùng tương đương – EP (tt)
 - Tuy nhiên nếu ta nhập vào **số thập phân** (55.5) hay một **ký tự không phải là số** (abc)?
 - Trong trường hợp trên có thể chia làm 5 phân vùng như sau:
 - Các số nguyên từ 1 đến 100
 - Các số nguyên nhỏ hơn 1
 - Các số nguyên lớn hơn 100
 - Không phải số
 - Số thập phân
 - Như vậy, việc phân vùng có đúng và đủ hay không là tùy thuộc vào kinh nghiệm của tester.



Các kỹ thuật trong kiểm thử hộp đen

- Kỹ thuật phân tích giá trị giới hạn - BVA (Boundary Value Analysis)
 - Kỹ thuật BVA sẽ chọn các giá trị nằm tại các điểm giới hạn của phân vùng.



- Áp dụng kỹ thuật BVA cần 4 test case để test trường hợp này: 0,1,100,101



Kỹ thuật EP & BVA

- Xét ví dụ: Một ngân hàng trả lãi cho khách hàng dựa vào số tiền còn lại trong tài khoản. Nếu số tiền từ 0 đến 100\$ thì trả 3% lãi, từ lớn hơn 100 \$ đến nhỏ hơn 1000\$ trả 5% lãi, từ 1000\$ trở lên trả 7% lãi.

– Dùng kỹ thuật EP:

Invalid partition	Valid (for 3% interest)		Valid (for 5%)		Valid (for 7%)
-\$0.01	\$0.00	\$100.00	\$100.01	\$999.99	\$1000.00

- Kỹ thuật EP: -0.44, 55.00, 777.50, 1200.00
- Kỹ thuật BVA: -0.01, 0.00, 100.00, 100.01, 999.99, 1000.00



Tại sao phải kết hợp BVA và EP

- Mỗi giá trị giới hạn đều nằm trong một phân vùng nào đó. Nếu chỉ sử dụng giá trị giới hạn thì ta cũng có thể test luôn phân vùng đó.
- Tuy nhiên vấn đề đặt ra là nếu như giá trị đó sai thì nghĩa là giá trị giới hạn bị sai hay là cả phân vùng bị sai. Hơn nữa, nếu chỉ sử dụng giá trị giới hạn thì không đem lại sự tin tưởng cho người dùng vì chúng ta chỉ sử dụng những giá trị đặc biệt thay vì sử dụng giá trị thông thường.
- Vì vậy, cần phải kết hợp cả BVA và EP



Ví dụ

Customer Name

2-64 chars.

Account number

6 digits, 1st non-zero

Loan amount requested

£500 to £9000

☐ Term of loan

☐ Monthly repayment

1 to 30 years

Term:

Repayment:

Interest rate:

Total paid back:

Minimum £10

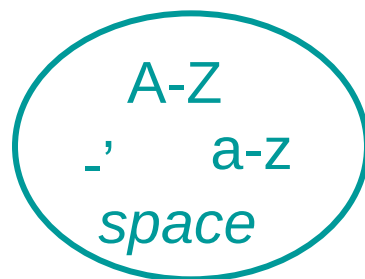


Customer name

Number of characters:



Valid characters:



Any
other

Conditions	Valid Partitions	Invalid Partitions	Valid Boundaries	Invalid Boundaries
Customer name	2 to 64 chars valid chars	< 2 chars > 64 chars invalid chars	2 chars 64 chars	1 chars 65 chars 0 chars



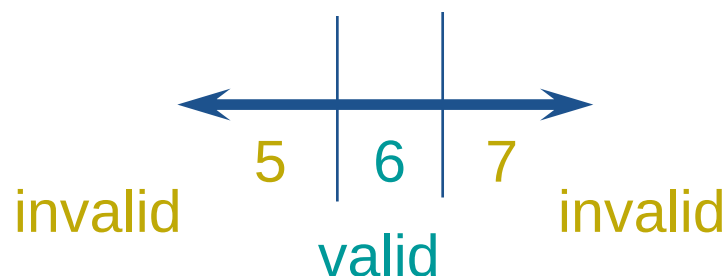
Account number

first character:

valid: non-zero

invalid: zero

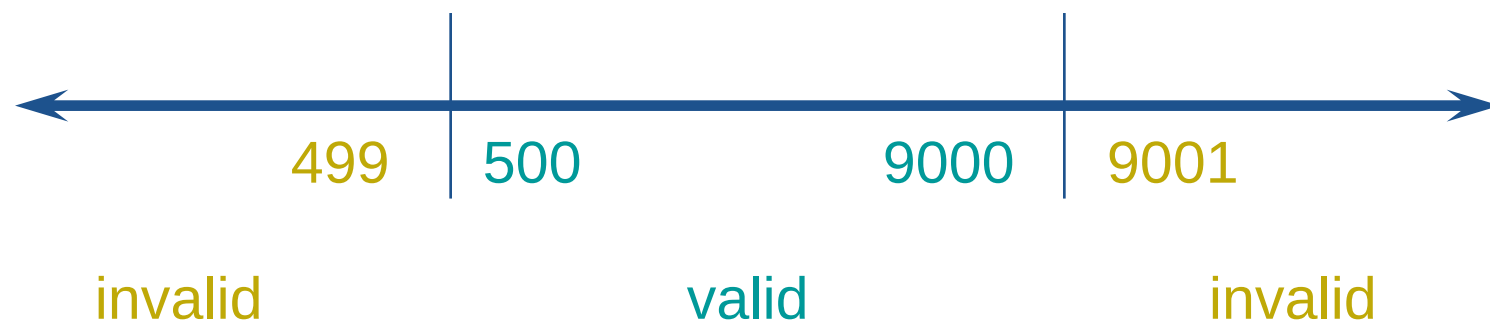
number of digits:



Conditions	Valid Partitions	Invalid Partitions	Valid Boundaries	Invalid Boundaries
Account number	6 digits 1 st non-zero	< 6 digits > 6 digits 1 st digit = 0 non-digit	100000 999999	5 digits 7 digits 0 digits



Loan amount



Conditions	Valid Partitions	Invalid Partitions	Valid Boundaries	Invalid Boundaries
Loan amount	500 - 9000	< 500 >9000 0 non-numeric null	500 9000	499 9001

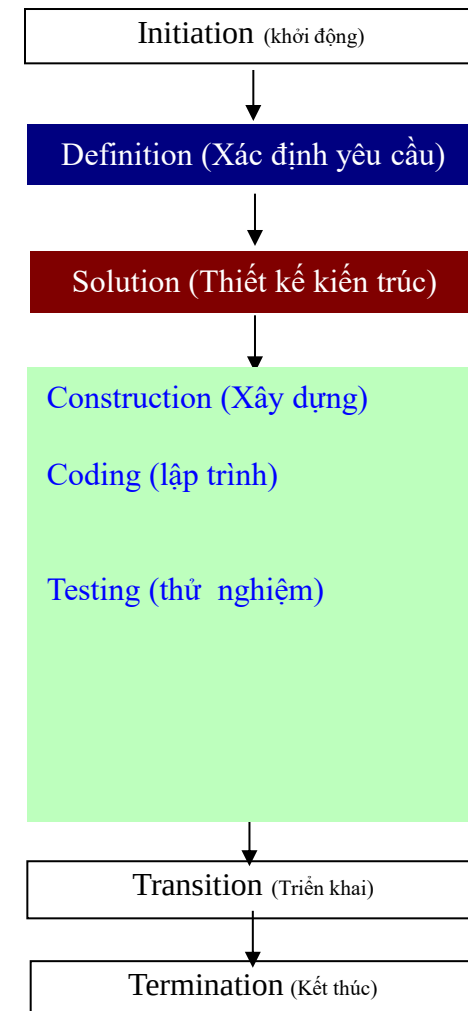
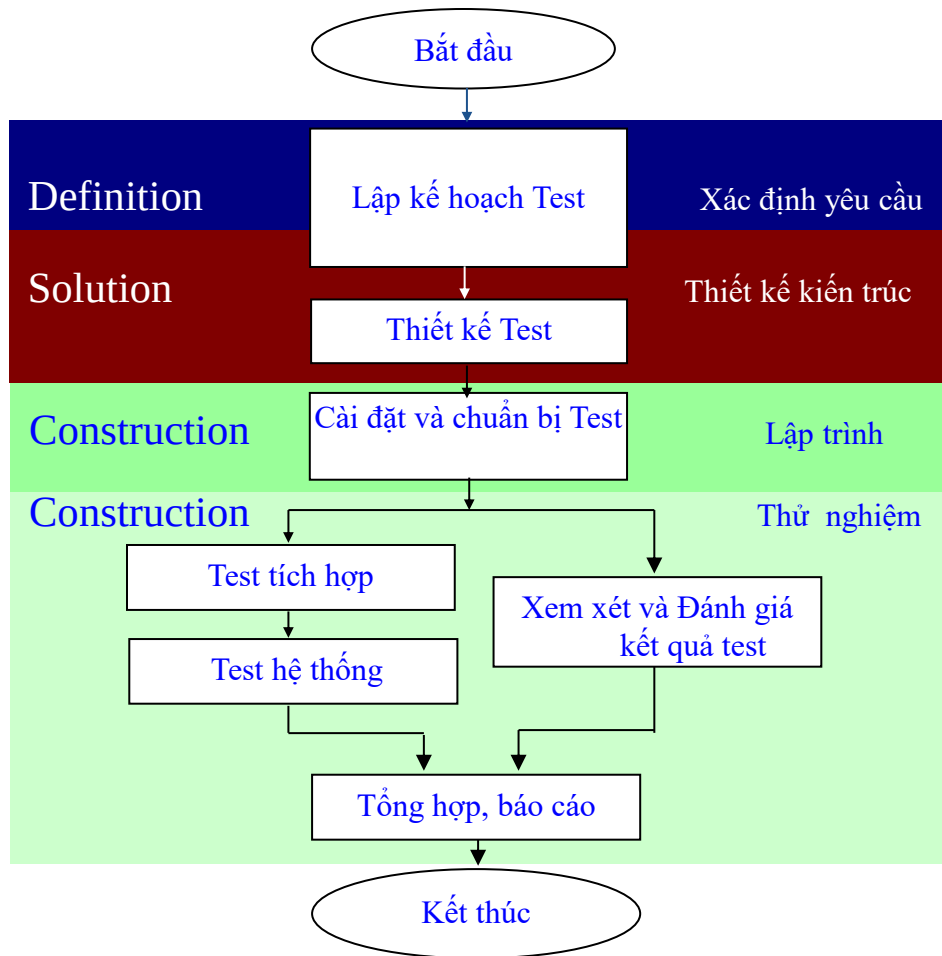


Condition template

Conditions	Valid Partitions	Tag	Invalid Partitions	Tag	Valid Boundaries	Tag	Invalid Boundaries	Tag
Customer name	2 - 64 chars valid chars	V1	< 2 chars	X1	2 chars 64 chars	B1	1 char	D1
		V2	> 64 chars	X2		B2	65 chars	D2
			invalid char	X3			0 chars	D3
Account number	6 digits 1 st non-zero	V3	< 6 digits	X4	100000 999999	B3	5 digits	D4
		V4	> 6 digits	X5		B4	7 digits	D5
			1 st digit = 0	X6			0 digits	D6
			non-digit	X7				
Loan amount	500 - 9000	V5	< 500	X8	500 9000	B5	499	D7
			> 9000	X9		B6	9001	D8
			0	X10				
			non-integer	X11				
			null	X12				

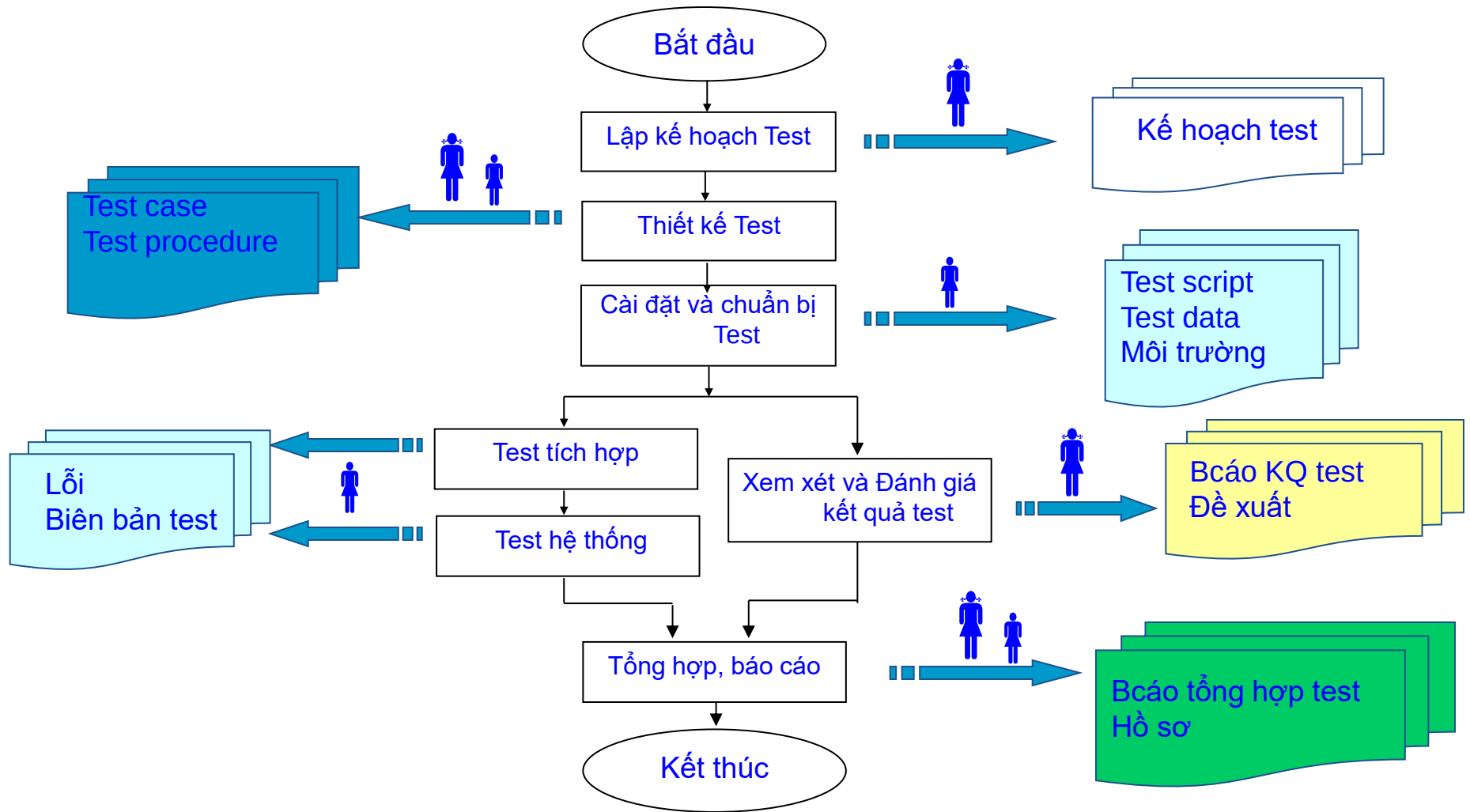


CÁC HOẠT ĐỘNG KIỂM THỬ





CÁC HOẠT ĐỘNG KIỂM THỬ





CÁC HOẠT ĐỘNG - Lập kế hoạch test

1. Xác định yêu cầu cho test

Các yêu cầu

- Dựa trên các sản phẩm phân tích
- Xác định: test cái gì?
- Xác định: giới hạn công việc, thời gian, nguồn lực

2. Đánh giá rủi ro và lập mức ưu tiên cho các yêu cầu

Các rủi ro

- Phương thức thực hiện
- Tiêu chuẩn hoàn thành việc test, đánh giá chất lượng sản phẩm
- Các yếu tố cần chú ý

3. Xác lập chiến lược test

TN Test

4. Xác định nguồn lực và môi trường

- Số người, kỹ năng
- Môi trường test: phần mềm, cứng...
- Công cụ test
- Dữ liệu test

5. Lập lịch trình test

TN Test

6. Tổng hợp thông tin, lập KH test

TN Test

7. Xem xét và thông qua KH test

- Xác định nguồn lực
- Lịch trình test
- Các mốc chính

QTDA



CÁC HOẠT ĐỘNG - thiết kế test

1. Lập danh sách các loại test, đảm bảo cho việc xác lập tính đúng đắn & thỏa mãn yêu cầu của sản phẩm

Danh

- Dựa trên các sản phẩm của:
 - Phân tích
 - bước lập KH test
- Xác định các test case
- Mô tả test case: vào, ra, điều kiện

2. Xây dựng tình huống test (test case)

Thiết
dụng

Tình huống kiểm thử

- Kiểm tra 1 chức năng của chương trình
- Các giá trị đầu vào $\xrightarrow{\text{Các điều kiện thực hiện}}$ Kết quả dự kiến

3. Xây dựng và tổ chức thủ tục test (test procedure)

Các t

CBT

4. Xem xét tình huống test và thủ tục test, đánh giá tỷ lệ yêu cầu của khách hàng (hoặc tình huống sử dụng) sẽ được test dựa trên thiết kế test đã lập

Đi

- Dựa vào: các test case
- Xác định các test procedure
- Xác lập mối quan hệ và thứ tự giữa
 - Các test procedure với nhau
 - Các test procedure - Test cases

5. Thông qua thiết kế Test

Thi

Thủ tục kiểm thử

- Các chỉ dẫn để thiết lập, thực hiện và đánh giá kết quả test
- Cho 1 hoặc nhiều tình huống test (test case)



Xây dựng tình huống test (test case)

- Một Test Case có thể coi là một tình huống kiểm tra, được thiết kế để kiểm tra một đối tượng có thỏa mãn yêu cầu đặt ra hay không. Một Test Case thường bao gồm 3 phần cơ bản:
 - Mô tả: đặc tả các điều kiện cần có để tiến hành kiểm tra.
 - Nhập: đặc tả đối tượng hay dữ liệu cần thiết, được sử dụng làm đầu vào để thực hiện việc kiểm tra.
 - Kết quả mong chờ: kết quả trả về từ đối tượng kiểm tra, chứng tỏ đối tượng đạt yêu cầu.



Design test cases

Test Case	Description	Expected Outcome		New Tags Covered
1	Name: John Smith Acc no: 123456 Loan: 2500 Term: 3 years	Term:	3 years	V1, V2, V3, V4, V5
		Repayment:	79.86	
		Interest rate:	10%	
		Total paid:	2874.96	
2	Name: AB Acc no: 100000 Loan: 500 Term: 1 year	Term:	1 year	B1, B3, B5,
		Repayment:	44.80	
		Interest rate:	7.5%	
		Total paid:	537.60	



CÁC HOẠT ĐỘNG - Cài đặt và chuẩn bị test

Test script

- Các lệnh dùng để tự động hoá các thủ tục test
- Lập trình, sử dụng công cụ test tự động

Lập trình hoặc sinh tự động: = tool

- **Kiểm tra thử nghiệm**

1. Lập các test script để thực hiện các tình huống test/thủ tục test (nếu cần)

Test scripts

TN Test/
CBT

2. Chuẩn bị dữ liệu test

- **Tạo mới dữ liệu**
- **Tái sử dụng các dữ liệu cũ**

CBT

3. **Chuẩn bị môi trường**

CBT

4. Kiểm tra các công cụ test

Biên bản kiểm tra công cụ test

CBT

5. Xem xét môi trường, điều kiện và dữ liệu test

Môi trường và dữ liệu test
Kiểm tra

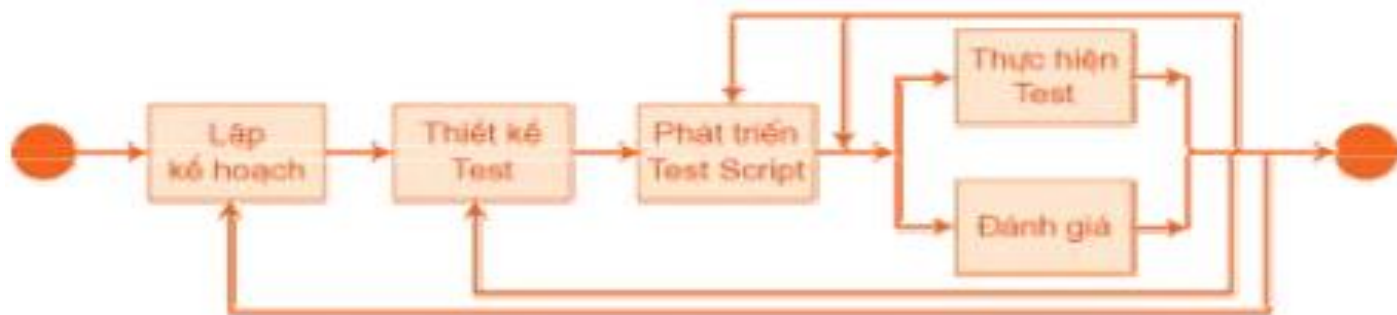
TN Test/
CBT

- **Phần mềm, phần cứng**
- **Các điều kiện khác**



Test Script

- Test Script là một nhóm mã lệnh dạng đặc tả kịch bản dùng để tự động hóa một trình tự kiểm tra, giúp cho việc kiểm tra nhanh hơn, hoặc cho những trường hợp mà kiểm tra bằng tay sẽ rất khó khăn hoặc không khả thi.
- Các Test Script có thể tạo thủ công hoặc tạo tự động dùng công cụ kiểm tra tự động.





CÁC HOẠT ĐỘNG - test tích hợp

		• Tài liệu • Gói phần mềm • ...
1. Nhận bàn giao với đội lập trình	Các sản phẩm cần nhận	
2. Cài đặt	Hệ thống test	• Dựa trên thiết kế test • Ghi nhận lỗi vào DMS
3. Thực hiện test và ghi nhận lỗi	Biên bản test	CBT
4. Xử lý lỗi	Danh sách lỗi phát hiện	TN Test, CBT
5. Xem xét các kết quả test và việc thực hiện khắc phục lỗi	Biên bản test	TN Test, QTDA, CBT, CBCL (nếu cần)
	• Phối hợp với đội lập trình • Test lại -> Ghi nhận lỗi mới	



CÁC HOẠT ĐỘNG - test hệ thống

1. Nhận bàn giao với đội lập trình	Các sản phẩm cần test được tiếp	CBT
2. Chỉnh sửa thiết kế test	• Kiểm tra và cập nhật: • Test case • Test procedure • Test script • Test data	TN Test
3. Cài đặt		CBT
4. Thực hiện test và ghi nhận lỗi		CBT
Danh sách lỗi phát hiện		
5. Xử lý lỗi	Biên bản test	TN Test, CBT
6. Xem xét các kết quả test thực hiện khắc phục lỗi	• Khi hệ thống thoả mãn các tiêu chuẩn đặt ra	TN Test,QTDA, CBT, CBCL (nếu cần)
7. Kết quả test được xem xét		Xác nhận sản phẩm đủ tiêu chuẩn phát hành



CÁC HOẠT ĐỘNG - xem xét và đánh giá kết quả test

1. Phân tích lỗi và đưa ra đề xuất	Báo cáo kết quả test Đề xuất	TN Test
2. Đánh giá tỷ lệ test, đánh giá mức độ đạt được các tiêu chí để hoàn thành test	Báo cáo kết quả test	TN Test
3. Xem xét báo cáo kết quả test	Báo cáo kết quả test được xem xét	QTDA, CBCL

- Dựa trên tỷ lệ YC được test/tổng số YC cần test

- Phân tích lỗi dựa:
 - mức độ lặp lại
 - độ nghiêm trọng
 - Thời gian sửa lỗi...



CÁC HOẠT ĐỘNG - tổng hợp, báo cáo

1. Tập hợp các dữ liệu, kết quả test	Dữ liệu, kết quả test	CBT
2. Lập báo cáo tổng hợp test	Báo cáo tổng hợp test	TN Test
3. Tổ chức lưu trữ tài liệu, hồ sơ	Hồ sơ, files	CBT



Thảo luận

